

Persistent Update-Only Evaluation Across Real Graphs

July 31, 2026

Question and protocol. This experiment asks whether Spine has an update-throughput advantage when graph computation is disabled (the equivalent of `max_iter=0`). Both architectures retain graph state across batches. Spine receives only the initial graph and each arriving batch. The official-style GraSU baseline is future-aware: it receives the complete update trace, performs update-density reordering once, and reserves PMA slots for that trace. We therefore report two distinct metrics: (1) accelerator device cycles for structure updates only, and (2) finite-trace time including measured CPU preprocessing plus a common parameterized 12 GB/s H2D and 10 μ s launch/synchronization cost per batch. No SSSP, PageRank, CC, convergence loop, or ReGraph compute stage runs.

All admitted rows pass every per-batch and final graph-state check. The primary cross-dataset experiment uses ten persistent insertion batches of 4,096 records each. Host timing is the median of three runs on AU, SU, and WK; memory-heavy SO and PK use one exploratory host run and are marked as such in the evidence.

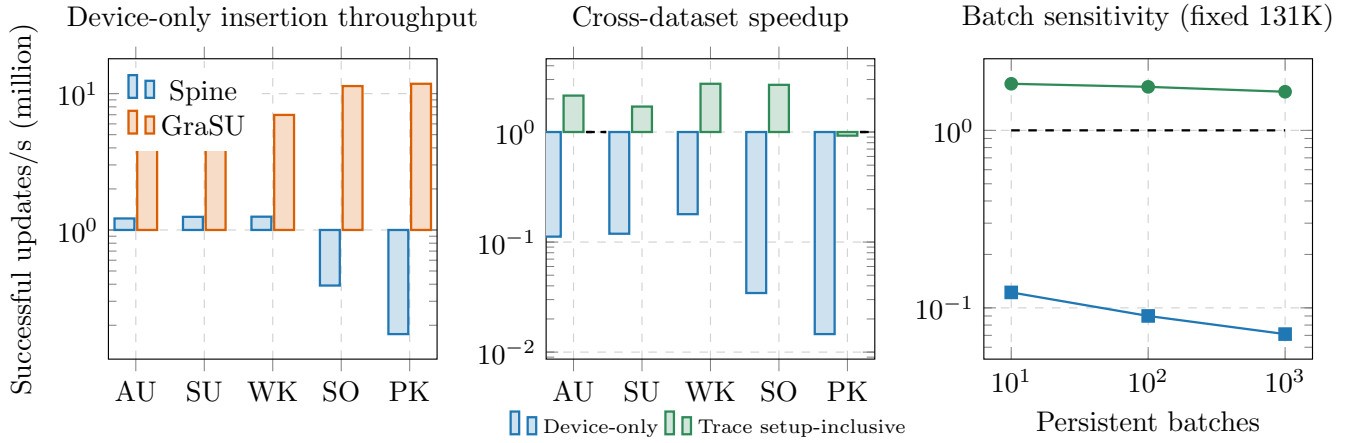


Figure 1: Persistent update-only results. Left: pure device insertion throughput. Center: Spine/GraSU speedup for device-only and finite-trace setup-inclusive time; values above one favor Spine. Right: the same 131,072 AU records split into 10, 100, and 1,000 persistent batches, isolating per-batch overhead rather than trace-length amortization.

Graph	Vertices	Initial edges	Spine Mupd/s	GraSU Mupd/s	Device speedup	Setup incl. speedup
AU	515,281	390,847	1.215	10.845	0.112×	2.143×
SU	567,316	616,118	1.248	10.484	0.119×	1.704×
WK	1,140,149	1,142,352	1.251	6.982	0.179×	2.744×
SO	6,024,271	23,724,166	0.391	11.364	0.034×	2.683×
PK	1,632,803	44,603,928	0.172	11.815	0.015×	0.927×

Table 1: Insertion results for 40,960 successful updates in ten batches.

Result. GraSU’s PMA device path is faster on every graph, by 5.6×–68.6×. The performance gap widens on SO and PK because Spine issues substantially more backend requests (171,943 vs. 21,779 on SO and 529,909 vs. 16,571 on PK). Thus the current evidence does *not* support a Spine update-kernel-throughput claim. For a short ten-batch known trace, including preprocessing reverses the ranking on AU, SU, WK, and SO because GraSU pays its one-time

reorder/PMA-build cost; PK is already an exception, with GraSU about $1.08\times$ faster even under that setup-inclusive metric. Setup-inclusive results are finite-trace amortization results, not device-throughput results.

Batch and operation sensitivity. With a fixed 131,072-record AU trace, increasing the batch count from 10 to 1,000 reduces Spine’s device speedup from $0.122\times$ to $0.071\times$ and its setup-inclusive speedup from $1.830\times$ to $1.650\times$. More launch boundaries hurt Spine; this sweep must not be interpreted as increasing trace length. On AU, deletion gives 1.472 Mupd/s for Spine and 9.903 Mupd/s for GraSU, close to the insertion result (1.385 vs. 10.278 Mupd/s). Weight increases are excluded: the frozen Spine differential merge retains the minimum weight, so that operation requires the separately evaluated full-graph fallback rather than this pure update-only path.

Scope. The simulator uses the common SST memHierarchy/DRAMSim3 backend and finite FIFO/AXI paths. The 12 GB/s transfer and 10 μ s launch terms are shared parameters, not measured XRT timings. The Bitcoin full graph is excluded by architecture capacity (24,575,382 vertices exceeds Spine’s frozen 2^{24} -vertex bound), not by timeout. Future-unknown online GraSU rebalancing is a separate baseline and is not silently substituted here.